

Agenda de contactos de la Quantum Wallet

Operaciones CRUD sobre SQLite integradas a Quantum Core

Implementación de una agenda profesional de contactos y de las operaciones de creación, lectura, actualización y eliminación desde Python, con validación en dos capas, controles de seguridad e integración con las transferencias de la billetera

ESTUDIANTE	Deibis Zuluaga Baena
NÚCLEO	Fundamentos de Software · FUNSO_2_N_H
ACTIVIDAD	Operaciones CRUD (Create, Read, Update, Delete) en Quantum Core
HERRAMIENTAS	SQLite 3 · Python 3.14 (sqlite3, logging) · DB Browser for SQLite
ENTREGABLES	practica_crud.py, quantum_wallet.db, crud_contactos.log, diagrama de integración (PNG, SVG y .drawio) e informe
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
FECHA DE ENTREGA	4 de octubre de 2026

16 páginas · 11 figuras

Objetivo del trabajo

Objetivo: realizar las operaciones CRUD (Create, Read, Update, Delete) sobre una nueva tabla de Quantum Core, `contactos`, vinculada a los usuarios mediante una llave foránea, sin alterar la estructura principal de la base de datos, e implementar cada operación como una función del script `practica_crud.py`.

CÓDIGO ORIGINAL DE LA ACTIVIDAD

El enunciado de la actividad y el repositorio del curso definen la tabla `contactos` (identificador, apodo, número y llave foránea al usuario) y cuatro operaciones: insertar cinco contactos para el usuario 1, mostrar en consola sus contactos, cambiar el apodo de un contacto mediante una sentencia con `WHERE` y eliminar el contacto con identificador 2. Esos requisitos se conservan. Para enriquecer el ejercicio, la tabla se convirtió en una agenda profesional con atributos adicionales (nombre completo, categoría, empresa, correo y ciudad), la carga inicial se amplió a diez contactos del entorno laboral y personal del usuario, y se agregaron restricciones, controles de seguridad y funciones complementarias, identificados en el código con el comentario `[AGREGADO]`.

BASE DE DATOS DE PARTIDA

La práctica opera sobre `quantum_wallet.db`, generada por `configurar_db.py` con las tablas de usuarios, billeteras y transacciones de Quantum Core. Para que el proyecto pueda ejecutarse de forma independiente, la carpeta incluye una copia de ese script. La tabla `contactos` se agrega sin modificar las tablas existentes.

ALCANCE DE LA SOLUCIÓN

Además de las cuatro operaciones solicitadas, la solución incorpora lectura filtrada por categoría y un resumen de la agenda por categoría, validación en Python y en la base de datos, autorización por usuario en las actualizaciones y eliminaciones, una lista blanca de campos editables, búsqueda de contactos, marcación de favoritos, transferencias de dinero a un contacto que tiene cuenta en Quantum Wallet, registro de operaciones en consola y en archivo, y un conjunto de pruebas que comprueba que las operaciones inválidas son rechazadas.

Estructura de la tabla contactos

La tabla almacena la agenda de cada usuario. Las filas con fondo amarillo corresponden a los campos adicionados para enriquecer el ejercicio.

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_contacto	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único del contacto.
id_usuario	INTEGER	NOT NULL; FK → usuarios ON DELETE CASCADE	Dueño de la agenda.
apodo	TEXT	NOT NULL; COLLATE NOCASE; 1 a 40 caracteres; único por agenda	Nombre con el que el usuario identifica al contacto.
nombre_completo	TEXT	NOT NULL; 3 a 80 caracteres	Nombre real de la persona o de la entidad.
categoria	TEXT	NOT NULL; CHECK: FAMILIA, TRABAJO, PROVEEDOR, CLIENTE, SERVICIO o EMERGENCIA	Clasificación del contacto para filtrar y resumir la agenda.
empresa	TEXT	Opcional	Organización a la que pertenece el contacto.
numero	TEXT	NOT NULL; solo dígitos; 3 a 15 caracteres; único por agenda	Número telefónico del contacto.
email	TEXT	CHECK: formato de correo	Correo electrónico opcional.
ciudad	TEXT	Opcional	Ubicación del contacto.
id_usuario_contacto	INTEGER	FK → usuarios ON DELETE SET NULL; distinto de id_usuario	Cuenta en Quantum Wallet del contacto, cuando la tiene.
favorito	INTEGER	NOT NULL; DEFAULT 0; CHECK: 0 o 1	Marca de contacto frecuente.
fecha_creacion	TEXT	NOT NULL; DEFAULT fecha actual	Fecha de registro del contacto.
fecha_actualizacion	TEXT	Se asigna en cada actualización	Trazabilidad de la última modificación.

Decisiones de diseño

DECISIÓN	JUSTIFICACIÓN
Número como texto	El número es un identificador, no una cantidad: se almacena como <code>TEXT</code> para conservar ceros iniciales y admitir líneas cortas como la de emergencias (123). La restricción <code>NOT GLOBAL UNIQUE</code> exige que contenga únicamente dígitos, y el límite de 15 corresponde a la longitud máxima de un número telefónico internacional.
Unicidad por agenda	Las restricciones <code>UNIQUE (id_usuario, numero)</code> y <code>UNIQUE (id_usuario, apodo)</code> impiden duplicados dentro de una misma agenda, pero permiten que dos usuarios registren el mismo número.
Apodo sin distinción de mayúsculas	Con <code>COLLATE NOCASE</code> , «Contador» y «CONTADOR» se consideran el mismo apodo. Esto evita ambigüedades al buscar un contacto o transferirle dinero por su apodo.

Dos llaves foráneas hacia usuarios

`id_usuario` indica de quién es la agenda; `id_usuario_contacto` indica si el contacto también es cliente de Quantum Wallet. La segunda es opcional, porque no todos los contactos tienen cuenta.

Creación limpia de la tabla

El script elimina y vuelve a crear la tabla en cada ejecución. Al eliminarla, SQLite reinicia también su contador `AUTOINCREMENT`, de modo que el contacto 2 corresponde siempre al mismo registro y las operaciones del enunciado son reproducibles.

Categorías controladas

La categoría solo admite seis valores, definidos en la restricción `CHECK` y en la constante `CATEGORIAS` de Python. Un conjunto cerrado evita variantes del mismo valor («Proveedor», «proveedores») y permite filtrar y resumir la agenda de forma confiable.

Índice por usuario y categoría

`idx_contactos_usuario` (`id_usuario`, `categoría`) acelera las consultas más frecuentes: leer la agenda de un usuario y filtrarla por categoría.

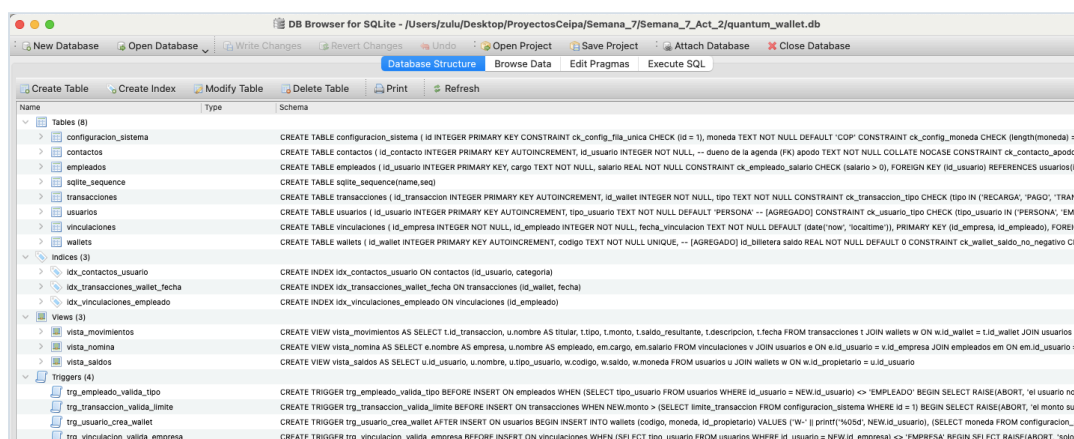


Figura 1. Estructura de `quantum_wallet.db` en DB Browser for SQLite: la tabla `contactos` y su índice se suman a las tablas existentes de Quantum Core.

Relación de la tabla contactos con el sistema

Quantum Wallet · Integración de la tabla contactos

Relación de la agenda de contactos con usuarios, billeteras y transacciones de quantum_wallet.db

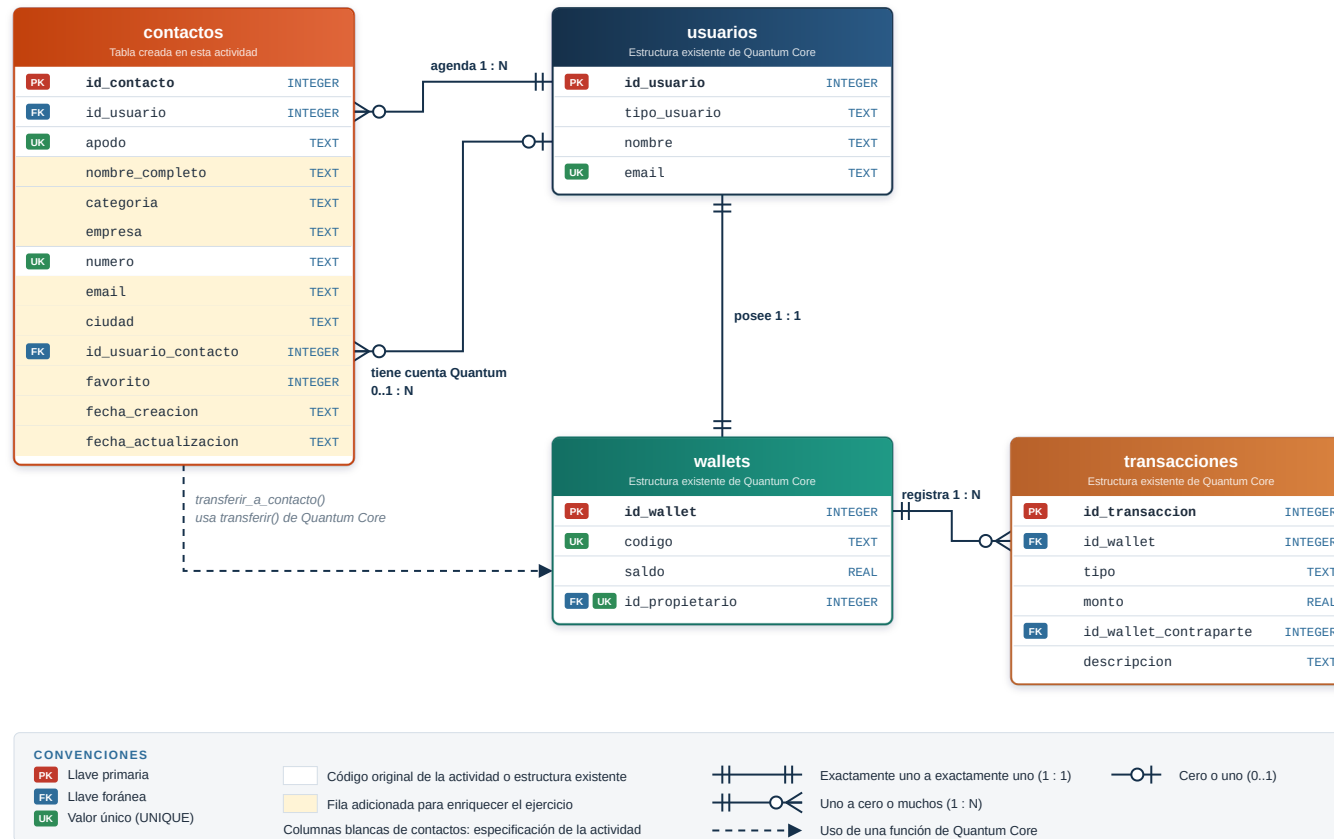


Figura 2. Integración de la tabla contactos con usuarios, billeteras y transacciones, en notación de pata de gallo. Elaborado en formato .drawio, editable con Draw.io Integration para Visual Studio Code.

Comportamiento de las relaciones

La tabla se integra a Quantum Core mediante dos llaves foráneas hacia `usuarios` y mediante la reutilización de las funciones del sistema para mover dinero.

RELACIÓN	CARDINALIDAD	COMPORTAMIENTO
usuarios – contactos (agenda)	1 : N	Cada contacto pertenece a un usuario. Si el usuario se elimina, su agenda se elimina con él (<code>ON DELETE CASCADE</code>).
usuarios – contactos (cuenta Quantum)	0..1 : N	Un contacto puede corresponder a otro cliente de Quantum Wallet. Si ese cliente se elimina, el contacto se conserva y la referencia queda nula (<code>ON DELETE SET NULL</code>).
contactos – wallets – transacciones	—	La función <code>transferir_a_contacto()</code> ubica al contacto por su apodo y ejecuta <code>transferir()</code> de <code>configurar_db.py</code> . La transferencia queda registrada en <code>transacciones</code> y respeta las reglas del sistema: saldo suficiente, límite por transacción y atomicidad del débito y el crédito.

INTEGRACIÓN SIN DUPLICAR REGLAS

La agenda no maneja dinero por su cuenta: delega la transferencia en la misma función que usa el resto del sistema. Así, una transferencia hecha desde la agenda queda sujeta a las restricciones `CHECK` y a los triggers que ya protegen a Quantum Core, y sus movimientos se registran en `transacciones` igual que cualquier otra operación del sistema.

Las operaciones CRUD en practica_crud.py

Cada operación del enunciado se implementó en una función propia y documentada. Todas usan parámetros `?`, y las operaciones de escritura se ejecutan dentro de una transacción.

OPERACIÓN	FUNCIÓN	SENTENCIA SQL	CONTROL APLICADO
C · Create	<code>crear_contactos()</code>	<code>INSERT</code> con <code>executemany</code>	Valida todas las filas antes de insertar; las guarda en una sola transacción: entran todas o ninguna.
R · Read	<code>leer_contactos()</code>	<code>SELECT ... LEFT JOIN usuarios ... WHERE id_usuario = ?</code>	Devuelve solo la agenda del usuario indicado, con filtro opcional por categoría, y agrega el nombre del contacto en Quantum Wallet cuando tiene cuenta.
U · Update	<code>actualizar_apodo()</code>	<code>UPDATE ... WHERE id_contacto = ? AND id_usuario = ?</code>	Modifica una sola fila, solo si pertenece al usuario, y verifica que la sentencia haya afectado exactamente un registro.
D · Delete	<code>borrar_contacto()</code>	<code>DELETE ... WHERE id_contacto = ? AND id_usuario = ?</code>	Comprueba que el contacto exista y pertenezca al usuario; elimina una fila, nunca la tabla, y registra qué se eliminó.

Funciones adicionales

FUNCIÓN	PROPÓSITO
<code>actualizar_contacto()</code>	Actualiza varios campos a la vez, solo si están en la lista blanca <code>CAMPOS_EDITABLES</code> .
<code>resumen_por_categoria()</code>	Cuenta los contactos de la agenda por categoría con <code>GROUP BY</code> .
<code>buscar_contactos()</code>	Busca por apodo, nombre completo, empresa o número con <code>LIKE</code> ; escapa los comodines <code>%</code> y <code>_</code> que escriba el usuario.
<code>transferir_a_contacto()</code>	Transfiere dinero a un contacto con cuenta en Quantum Wallet, por su apodo.
<code>validar_apodo()</code> , <code>validar_nombre()</code> , <code>validar_categoria()</code> , <code>validar_numero()</code> , <code>validar_email()</code> , <code>validar_id()</code>	Primera capa de validación, previa a cualquier sentencia SQL.
<code>configurar_log()</code>	Registra cada operación en la consola y en <code>crud_contactos.log</code> .
<code>probar_seguridad()</code>	Ejecuta las pruebas de inyección SQL y de operaciones inválidas.

Actualización y eliminación con WHERE

```
# ----- U : UPDATE -----
with conexion:
    cursor = conexion.execute(
        "UPDATE contactos SET apodo = ?, fecha_actualizacion = datetime('now', 'localtime') "
        "WHERE id_contacto = ? AND id_usuario = ?;",
        (nuevo_apodo, id_contacto, id_usuario),
    )
    if cursor.rowcount == 0:
        raise LookupError(f"el contacto {id_contacto} no existe en la agenda del usuario {id_usuario}")

# ----- D : DELETE -----
with conexion:
    cursor = conexion.execute(
        "DELETE FROM contactos WHERE id_contacto = ? AND id_usuario = ?;",
        (id_contacto, id_usuario),
    )
```

La condición combina la llave primaria con el dueño de la agenda. La primera parte garantiza que se afecta una sola fila; la segunda impide que un usuario modifique o elimine contactos de otro. Una sentencia `UPDATE` o `DELETE` sin `WHERE` afectaría a todas las filas de la tabla.

Documentación del código

ELEMENTO	CONTENIDO
Encabezado del script	Propósito del archivo, requisitos del enunciado que se conservan, elementos adicionados, dependencia de <code>configurar_db.py</code> e instrucciones de ejecución, con y sin pausas.
Docstrings	Cada función describe qué hace, qué valida y qué garantiza; por ejemplo, que el <code>WHERE</code> combina la llave primaria con el dueño de la agenda.
Comentarios en el esquema	Las restricciones principales llevan un comentario con la regla que protegen. Las secciones del código se separan por operación (C, R, U, D), y los elementos nuevos llevan la marca <code>[AGREGADO]</code> .
Restricciones con nombre	<code>ck_contacto_numero</code> , <code>uq_contacto_apodo</code> y las demás identifican en el propio mensaje de error la regla que se incumplió.
README	Estructura de la tabla, operaciones, controles de seguridad, instrucciones de ejecución y salida esperada.
Registro de operaciones	<code>crud_contactos.log</code> conserva, con fecha y hora, lo leído, actualizado, eliminado y rechazado en cada ejecución.

Controles sobre la tabla contactos

Las reglas se aplican en dos capas: Python valida los datos antes de construir la sentencia, y la base de datos las vuelve a exigir con restricciones. Si un programa omite la validación de Python e inserta directamente con SQL, la base de datos rechaza igualmente los datos inválidos.

REGLA	PRIMERA CAPA: PYTHON	SEGUNDA CAPA: BASE DE DATOS
Apodo de 1 a 40 caracteres	<code>validar_apodo()</code>	<code>CHECK ck_contacto_apodo</code>
Nombre completo de 3 a 80 caracteres	<code>validar_nombre()</code>	<code>CHECK ck_contacto_nombre</code>
Categoría permitida	<code>validar_categoria()</code>	<code>CHECK ck_contacto_categoria</code>
Número solo con dígitos	<code>validar_numero()</code>	<code>CHECK ck_contacto_numero</code>
Correo con formato válido	<code>validar_email()</code>	<code>CHECK ck_contacto_email</code>
Sin duplicados en la agenda	—	<code>UNIQUE uq_contacto_numero , uq_contacto_apodo</code>
Usuario existente	—	Llave foránea con <code>PRAGMA foreign_keys = ON</code>
No agendarse a sí mismo	—	<code>CHECK ck_contacto_no_propio</code>

Controles de seguridad

CONTROL	RIESGO QUE MITIGA
Consultas parametrizadas (?)	Inyección SQL: los datos del usuario se transmiten como valores y nunca se interpretan como código.
WHERE con llave primaria y dueño	Modificación masiva por error y acceso a contactos de otro usuario.
Verificación de filas afectadas	Operaciones que no encuentran el registro y fallan en silencio.
Lista blanca de campos	Cambios en columnas protegidas, como el dueño de la agenda, a través de la actualización genérica.
Identificadores enteros	Uso de textos arbitrarios como identificador en las operaciones de escritura.
Escape de comodines en LIKE	Búsquedas que devuelven más resultados de los esperados.
Transacciones atómicas	Cargas parciales o datos inconsistentes ante un error intermedio.
Registro de operaciones	Falta de trazabilidad sobre lo que se creó, modificó o eliminó.

Pruebas de inyección SQL

La lectura se ejecutó con el texto `1 OR 1=1` como identificador de usuario, y la búsqueda con `' OR '1'='1`. Ambas devolvieron cero filas: SQLite trató cada texto como un valor por comparar y no como parte de la sentencia, por lo que la condición no se alteró.

Operaciones inválidas

El script intenta trece operaciones que violan las reglas. Todas fueron rechazadas y revertidas.

OPERACIÓN INTENTADA	CAPA	RESULTADO
Número con letras (300-ABC)	Python	ValueError : número no válido
Apodo vacío	Python	ValueError : longitud del apodo
Categoría no permitida (AMIGO)	Python	ValueError : categoría no válida
Nombre completo demasiado corto	Python	ValueError : longitud del nombre
Número repetido en la agenda	Base de datos	UNIQUE constraint failed
Apodo repetido con otras mayúsculas	Base de datos	UNIQUE constraint failed
Contacto para un usuario inexistente	Base de datos	FOREIGN KEY constraint failed
Agendarse a sí mismo	Base de datos	CHECK constraint failed: ck_contacto_no_propio
Modificar un contacto de otro usuario	Autorización	LookupError : el contacto no está en la agenda
Borrar un contacto que ya no existe	Autorización	LookupError
Actualizar el dueño de la agenda	Lista blanca	ValueError : campo no permitido
SQL directo, sin validación de Python	Base de datos	CHECK constraint failed: ck_contacto_categoria
Transferir a un contacto sin cuenta	Integración	ValueError : no tiene cuenta en Quantum Wallet

Evidencias de las operaciones CRUD

Después de crear la base con `configurar_db.py`, el script se ejecutó con `python3 practica_crud.py --pausas`. La opción `--pausas` detiene la ejecución después de cada operación de escritura, de modo que el estado de la tabla puede verificarse en DB Browser for SQLite antes y después de cada cambio.

C · Create

```

QUANTUM CORE - Operaciones CRUD sobre la agenda de contactos

INFO | Tabla contactos creada: 14 columnas, 2 FK, 7 CHECK, 2 UNIQUE y 1 indice

--- C - CREATE ---
INFO | C - Create: 10 contactos insertados para el usuario 1
INFO | C - Create: 1 contacto insertado para el usuario 4
INFO | C - Create: 1 contacto insertado para el usuario 3

>>> Captura C: en DB Browser abra la tabla contactos (Browse Data, Refresh).
>>> Presione Enter para continuar...

```

Figura 3. Consola: creación de la tabla e inserción de diez contactos para el usuario 1 y de un contacto para cada uno de los usuarios 4 y 3.

	id_contacto	id_usuario	apodo	nombre_completo	categoria	empresa	numero	email	ciudad	id_usuario_contacto	favorito	fecha_creacion	fecha_
1	1	1	Contador	Andrés Felipe Restrepo Ospina	PROVEEDOR	Restrepo Contadores Asociados	31044589721	arestrepo@restrepocontadores.co	Medellín		0	2026-10-03 18:16:46	NULL
2	2	1	Antiguo arrendador	Jorge Iván Cardona Ruiz	SERVICIO	Inmobiliaria Los Andes	6045612835	NULL	Rionegro		0	2026-10-03 18:16:46	NULL
3	3	1	Jefe inmediato	Mariana Velásquez Henao	TRABAJO	Quantum SAS	3157725410	mvelasquez@quantum.com	Medellín		0	2026-10-03 18:16:46	NULL
4	4	1	Laure - Desarrolló	Laure Gomez	TRABAJO	Quantum SAS	3104567890	laure.gomez@quantum.com	Medellín	4	0	2026-10-03 18:16:46	NULL
5	5	1	Pedro - Analista	Pedro Rios	TRABAJO	Quantum SAS	3126665544	NULL	Medellín	6	0	2026-10-03 18:16:46	NULL
6	6	1	Cliente principal	Catalina Zuhaga Montoya	CLIENTE	Cerámica Viboral SAS	3005858214	compra@ceramicaviboral.co	El Carmen de Viboral		0	2026-10-03 18:16:46	NULL
7	7	1	Soporte de software	Nicolás Ramírez Echeverri	PROVEEDOR	Tecnoretro SAS	3126654098	soporte@tecnoretro.co	El Retiro		0	2026-10-03 18:16:46	NULL
8	8	1	Asesora jurídica	Daniela Ospina Giraldo	PROVEEDOR	Ospina & Asociados	3157745001	dospina@ospinaabogados.co	Medellín		0	2026-10-03 18:16:46	NULL
9	9	1	Hermano	Santiago Torres Arango	FAMILIA	NULL	3014456677	NULL	La Ceja		0	2026-10-03 18:16:46	NULL
10	10	1	Línea de emergencias	Línea Única de Emergencias	EMERGENCIA	NULL	123	NULL	Nacional		0	2026-10-03 18:16:46	NULL
11	11	4	Ana - Finanzas	Ana Torres	TRABAJO	Quantum SAS	3157778899	NULL	NULL		1	2026-10-03 18:16:46	NULL
12	12	3	Pedro Rios - Mimina	Pedro Rios	TRABAJO	NULL	3126665544	NULL	NULL	6	0	2026-10-03 18:16:46	NULL

Figura 4. Tabla contactos después del Create: doce registros con nombre completo, categoría, empresa, número, correo y ciudad; el identificador 1 corresponde al «Contador» y el 2 al «Antiguo arrendador».

R · Read y U · Update

```
-- R - READ
INFO | R - Read: 10 contactos del usuario 1
INFO | id apodo nombre completo categoria numero ciudad cuenta Quantum
INFO | 1 Contador Andrés Felipe Restrepo Ospina PROVEEDOR 3104458721 Medellín -
INFO | 2 Antiguo arrendador Jorge Iván Cardona Ruiz SERVICIO 6045612233 Rionegro -
INFO | 3 Jefe inmediato Mariana Velásquez Henao TRABAJO 3157723410 Medellín -
INFO | 4 Laura - Desarrollo Laura Gomez TRABAJO 3104567890 Medellín Laura Gomez
INFO | 5 Pedro - Analista Pedro Rios TRABAJO 3126654099 Medellín Pedro Rios
INFO | 6 Cliente principal Catalina Zuluaga Montoya CLIENTE 3005582214 El Carmen de Viboral -
INFO | 7 Soporte de software Nicolás Ramírez Echeverri PROVEEDOR 3126654098 El Retiro -
INFO | 8 Asesora jurídica Daniela Ospina Giraldo PROVEEDOR 3187742001 Medellín -
INFO | 9 Hermano Santiago Torres Arango FAMILIA 3014456677 La Ceja -
INFO | 10 Línea de emergencias Línea Única de Emergencias EMERGENCIA 123 Nacional -
INFO | Resumen de la agenda del usuario 1 por categorías: PROVEEDOR 3, TRABAJO 3, CLIENTE 1, EMERGENCIA 1, FAMILIA 1, SERVICIO 1
INFO | R - Read: 3 contactos del usuario 1 (categoría PROVEEDOR)
INFO | id apodo nombre completo categoria numero ciudad cuenta Quantum
INFO | 1 Contador Andrés Felipe Restrepo Ospina PROVEEDOR 3104458721 Medellín -
INFO | 7 Soporte de software Nicolás Ramírez Echeverri PROVEEDOR 3126654098 El Retiro -
INFO | 8 Asesora jurídica Daniela Ospina Giraldo PROVEEDOR 3187742001 Medellín -

-- U - UPDATE
INFO | U - Update: contacto 1 'Contador' -> 'Contador y revisor fiscal' (1 fila modificada)
INFO | U - Update: contacto 7 actualizado {'numero': '3126654099', 'email': 'mesadeayuda@tecnoretiro.co'}
INFO | U - Update: contacto 4 actualizado {'favorito': 1}

>>> Captura U: actualice DB Browser (Refresh) y revise los contactos 1, 4 y 7.
>>> Presione Enter para continuar...
```

Figura 5. Consola: lectura de los diez contactos del usuario 1, resumen por categoría, filtro de proveedores y las tres actualizaciones.

id_contacto	id_usuario	apodo	nombre_completo	categoria	empresa	numero	email	ciudad	id_usuario_contacto	favorito	fecha_creacion
1	1	Contador y revisor fiscal	Andrés Felipe Restrepo Ospina	PROVEEDOR	Restrepo Contadores Asociados	3104458721	arestrepo@restrepocontadores.co	Medellín		0	2026-10-03 18:16
2	1	Antiguo arrendador	Jorge Iván Cardona Ruiz	SERVICIO	Inmobiliaria Los Andes	6045612233		Rionegro		0	2026-10-03 18:16
3	1	Jefe inmediato	Mariana Velásquez Henao	TRABAJO	Quantum SAS	3157723410	mvelasquez@quantum.com	Medellín		0	2026-10-03 18:16
4	1	Laura - Desarrollo	Laura Gomez	TRABAJO	Quantum SAS	3104567890	laurea.gomez@quantum.com	Medellín	4	1	2026-10-03 18:16
5	1	Pedro - Analista	Pedro Rios	TRABAJO	Quantum SAS	3106665644		Medellín	8	0	2026-10-03 18:16
6	1	Cliente principal	Catalina Zuluaga Montoya	CLIENTE	Cerámicas Viboral SAS	3005582214	compras@ceramicasviboral.co	El Carmen de Viboral		0	2026-10-03 18:16
7	1	Soporte de software	Nicolás Ramírez Echeverri	PROVEEDOR	TecnoRetiro SAS	3126654099	mesadeayuda@tecnoretiro.co	El Retiro		0	2026-10-03 18:16
8	1	Asesora jurídica	Daniela Ospina Giraldo	PROVEEDOR	Ospina & Asociados	3187742001	dospina@ospinaabogados.co	Medellín		0	2026-10-03 18:16
9	1	Hermano	Santiago Torres Arango	FAMILIA		3014456677		La Ceja		0	2026-10-03 18:16
10	1	Línea de emergencias	Línea Única de Emergencias	EMERGENCIA		123		Nacional		0	2026-10-03 18:16
11	1	Ana - Finanzas	Ana Torres	TRABAJO	Quantum SAS	3157779899			1	0	2026-10-03 18:16
12	1	Pedro Rios - Mímina	Pedro Rios	TRABAJO		3106665644			8	0	2026-10-03 18:16

Figura 6. Tabla contactos después del Update: el contacto 1 cambió a «Contador y revisor fiscal», el contacto 7 registra la nueva línea y el nuevo correo de soporte, y el contacto 4 quedó marcado como favorito.

D · Delete

```
-- D - DELETE
INFO | D - Delete: contacto 2 'Antiguo arrendador' (Jorge Iván Cardona Ruiz, 6045612233) eliminado (1 fila)

>>> Captura D: actualice DB Browser (Refresh): el contacto 2 ya no está.
>>> Presione Enter para continuar...
```

Figura 7. Consola: eliminación del contacto 2 («Antiguo arrendador», Jorge Iván Cardona Ruiz), cuyo contrato de arrendamiento finalizó.

DB Browser for SQLite - /Users/zulu/Desktop/ProyectosCeipa/Semana_7/Semana_7_Act_2/quantum_wallet.db													
Write changes to the database file													
Database Structure													
Browse Data													
Edit Pragma													
Execute SQL													
Table: contactos													
id_contacto	id_usuario	apodo	nombre_completo	categoria	empresa	numero	email	ciudad	id_usuario_contacto	favorito	fecha_creacion		
Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter		
1	1	1 Contador y revisor fiscal	Andrés Felipe Restrepo Ospina	PROVEEDOR	Restrepo Contadores Asociados	3104458721	arestrepo@restrepocontadores.co	Medellín		0	2026-10-03 15:16:		
2	3	1 Jefe inmediato	Mariana Velásquez Henao	TRABAJO	Quantum SAS	3157723410	mvelasquez@quantum.com	Medellín		0	2026-10-03 15:16:		
3	4	1 Laura - Desarrollo	Laura Gomez	TRABAJO	Quantum SAS	3104667890	laura.gomez@quantum.com	Medellín		4	1	2026-10-03 15:16:	
4	5	1 Pedro - Analista	Pedro Rios	TRABAJO	Quantum SAS	3126665544		Medellín		6	0	2026-10-03 15:16:	
5	6	1 Cliente principal	Catalina Zuluaga Montoya	CLIENTE	Cerámicas Viboral SAS	3005582214	compres@ceramicaviboral.co	El Carmen de Viboral			0	2026-10-03 15:16:	
6	7	1 Soporte de software	Nicolás Ramírez Echeverri	PROVEEDOR	TecnoRetiro SAS	3126654099	mesadagrada@tecnoretiro.co	El Retiro			0	2026-10-03 15:16:	
7	8	1 Asesora jurídica	Daniela Ospina Giraldo	PROVEEDOR	Ospina & Asociados	3187742001	dospina@ospinaabogados.co	Medellín		0	2026-10-03 15:16:		
8	9	1 Hermano	Santiago Torres Arango	FAMILIA		3014456677		La Ceja			0	2026-10-03 15:16:	
9	10	1 Línea de emergencias	Línea Única de Emergencias	EMERGENCIA		123		Nacional			0	2026-10-03 15:16:	
10	11	4 Ana - Finanzas	Ana Torres	TRABAJO	Quantum SAS	3157778999				1	0	2026-10-03 15:16:	
11	12	3 Pedro Rios - Mimina	Pedro Rios	TRABAJO		3126665544				0	0	2026-10-03 15:16:	

Figura 8. Tabla contactos después del Delete: once registros; el identificador 2 ya no existe.

Funcionalidades adicionales, integración y pruebas

```

--- BUSQUEDA [AGREGADO] ---
INFO | Busqueda 'Quantum' en la agenda del usuario 1: 3 resultado(s) ['Jefe inmediato', 'Laura - Desarrollo', 'Pedro - Analista']

--- INTEGRACION CON QUANTUM CORE [AGREGADO] ---
INFO | Transferencia de 150,000.00 a 'Laura - Desarrollo' registrada en Quantum Core
INFO | Saldo de Ana Torres : 220,000.50 -> 70,000.50
INFO | Saldo de Laura Gomez : 4,349,999.50 -> 4,499,999.50

--- SEGURIDAD Y VALIDACION [AGREGADO] ---
INFO | Inyeccion SQL en la lectura: leer_contactos(conexion, '1 OR 1=1')
INFO | R - Read: 0 contactos del usuario 1 OR 1=1
INFO | -> 0 filas: el texto se trato como un valor, no como codigo SQL
INFO | Busqueda '1 OR 1=1' en la agenda del usuario 1: 0 resultado(s) []
INFO | -> la busqueda con comillas devolvio 0 filas
WARNING RECHAZADA Numero con letras -> ValueError: numero no valido '300-ABC': solo digitos, entre 3 y 15
WARNING RECHAZADA Apodo vacio -> ValueError: el apodo debe tener entre 1 y 40 caracteres
WARNING RECHAZADA Categoria no permitida -> ValueError: categoria no valida 'AMIGO': use FAMILIA, TRABAJO, PROVEEDOR, CLIENTE, SERVICIO, EMERGENCIA
WARNING RECHAZADA Nombre completo demasiado corto -> ValueError: el nombre completo debe tener entre 3 y 80 caracteres
WARNING RECHAZADA Numero repetido en la agenda -> IntegrityError: UNIQUE constraint failed: contactos.id_usuario, contactos.numero
WARNING RECHAZADA Apodo repetido (sin importar mayusculas) -> IntegrityError: UNIQUE constraint failed: contactos.id_usuario, contactos.apodo
WARNING RECHAZADA Contacto para un usuario inexistente -> IntegrityError: FOREIGN KEY constraint failed
WARNING RECHAZADA Agendarse a si mismo -> IntegrityError: CHECK constraint failed: ck_contacto_no_propio
WARNING RECHAZADA Modificar un contacto de otro usuario -> LookupError: el contacto 11 no existe en la agenda del usuario 1
WARNING RECHAZADA Borrar un contacto que ya no existe -> LookupError: el contacto 2 no existe en la agenda del usuario 1
WARNING RECHAZADA Actualizar un campo fuera de la lista blanca -> ValueError: campos no permitidos: ['id_usuario']
WARNING RECHAZADA SQL directo que evita la validacion de Python -> IntegrityError: CHECK constraint failed: ck_contacto_categoria
WARNING RECHAZADA Transferir a un contacto sin cuenta Quantum -> ValueError: el contacto 'Contador y revisor fiscal' no tiene cuenta en Quantum Wallet
INFO | Resultado: 13 de 13 operaciones invalidas rechazadas

--- ESTADO FINAL ---
INFO | R - Read: 9 contactos del usuario 1
INFO | id apodo nombre completo categoria numero ciudad cuenta Quantum
INFO | 1 Contador y revisor fiscal Andrés Felipe Restrepo Ospina PROVEEDOR 3104458721 Medellín -
INFO | 3 Jefe inmediato Mariana Velásquez Henao TRABAJO 3157723410 Medellín -
INFO | * 4 Laura - Desarrollo Laura Gomez TRABAJO 3104667890 Medellín Laura Gomez
INFO | 5 Pedro - Analista Pedro Rios TRABAJO 3126665544 Medellín Pedro Rios
INFO | 6 Cliente principal Catalina Zuluaga Montoya CLIENTE 3005582214 El Carmen de Viboral -
INFO | 7 Soporte de software Nicolás Ramírez Echeverri PROVEEDOR 3126654099 El Retiro -
INFO | 8 Asesora jurídica Daniela Ospina Giraldo PROVEEDOR 3187742001 Medellín -
INFO | 9 Hermano Santiago Torres Arango FAMILIA 3014456677 La Ceja -
INFO | 10 Línea de emergencias Línea Única de Emergencias EMERGENCIA 123 Nacional -
INFO | Resumen de la agenda del usuario 1 por categoria: PROVEEDOR 3, TRABAJO 3, CLIENTE 1, EMERGENCIA 1, FAMILIA 1

Registro completo en crud_contactos.log
zulu@MacBook-Air-de-Deibis Semana 7 Act 2 %

```

Figura 9. Consola: búsqueda por empresa, transferencia de 150.000 a Laura, pruebas de seguridad (13 de 13 rechazadas), estado final de la agenda y resumen por categoría.

id_transaccion	id_wallet	tipo	monto	saldo_resultante	id_wallet_contraparte	descripcion	fecha
1	1	RECARGA	100000.0	100000.0	NULL	Recarga	2026-10-03 15:16:46
2	2	PAGO	30000.0	70000.0	NULL	Pago en comercio	2026-10-03 15:16:46
3	3	RECARGA	800000.0	800000.0	NULL	Recarga	2026-10-03 15:16:46
4	4	PAGO	800000.0	300000.0	NULL	Pago a proveedor	2026-10-03 15:16:46
5	8	RECARGA	1000000.0	1000000.0	NULL	Capital de trabajo	2026-10-03 15:16:46
6	6	TRANSFERENCIA_ENVIADA	4800000.0	8800000.0	4	Pago de nomina	2026-10-03 15:16:46
7	7	TRANSFERENCIA_RECIBIDA	4800000.0	4800000.0	3	Pago de nomina	2026-10-03 15:16:46
8	8	TRANSFERENCIA_ENVIADA	3200000.0	2300000.0	8	Pago de nomina	2026-10-03 15:16:46
9	9	TRANSFERENCIA_RECIBIDA	3200000.0	3200000.0	3	Pago de nomina	2026-10-03 15:16:46
10	10	TRANSFERENCIA_ENVIADA	180000.5	4349999.5	1	Devolucion de prestamo	2026-10-03 15:16:46
11	11	TRANSFERENCIA_RECIBIDA	180000.5	220000.5	4	Devolucion de prestamo	2026-10-03 15:16:46
12	13	TRANSFERENCIA_ENVIADA	180000.0	70000.5	4	Reembolso de gastos de viaje (Laura - Desarrollo)	2026-10-03 15:26:15
13	14	TRANSFERENCIA_RECIBIDA	180000.0	4499999.5	1	Reembolso de gastos de viaje (Laura - Desarrollo)	2026-10-03 15:26:15

Figura 10. Tabla transacciones: el reembolso de gastos de viaje a Laura quedó registrado como débito en la billetera de Ana Torres y crédito en la de Laura Gomez (identificadores 13 y 14).

INTERPRETACIÓN

La transferencia redujo el saldo de Ana Torres de 220.000,50 a 70.000,50 e incrementó el de Laura Gomez de 4.349.999,50 a 4.499.999,50. El identificador de las transacciones pasa de 11 a 13 porque el número 12 lo utilizó el usuario temporal de la prueba de borrado en cascada de configurar_db.py ; AUTOINCREMENT no reutiliza identificadores.

```

Semana_7 > Semana_7_Act_2 > crud_contactos.log
1 2026-10-03 15:16:46 INFO Tabla contactos creada: 14 columnas, 2 FK, 7 CHECK, 2 UNIQUE y 1 indice
2 2026-10-03 15:16:46 INFO | C - Create: 10 contactos insertados para el usuario 1
3 2026-10-03 15:16:46 INFO | C - Create: 1 contacto insertado para el usuario 4
4 2026-10-03 15:16:46 INFO | C - Create: 1 contacto insertado para el usuario 3
5 2026-10-03 15:20:49 INFO | R - Read: 10 contactos del usuario 1
6 2026-10-03 15:20:49 INFO | id apodo nombre completo categoria numero ciudad cuenta Quantum
7 2026-10-03 15:20:49 INFO | 1 Contador Andrés Felipe Restrepo Ospina PROVEEDOR 3104458721 Medellín -
8 2026-10-03 15:20:49 INFO | 2 Antiguo arrendador Jorge Iván Cardona Ruiz SERVICIO 6045612233 Rionegro -
9 2026-10-03 15:20:49 INFO | 3 Jefe inmediato Mariana Velásquez Henao TRABAJO 3157723410 Medellín -
10 2026-10-03 15:20:49 INFO | 4 Laura - Desarrollo Laura Gomez TRABAJO 3104567890 Medellín Laura Gomez
11 2026-10-03 15:20:49 INFO | 5 Pedro - Analista Pedro Rios TRABAJO 3126665544 Medellín Pedro Rios
12 2026-10-03 15:20:49 INFO | 6 Cliente principal Catalina Zuluaga Montoya CLIENTE 3005582214 El Carmen de Viboral -
13 2026-10-03 15:20:49 INFO | 7 Soporte de software Nicolás Ramírez Echeverri PROVEEDOR 3126654098 El Retiro -
14 2026-10-03 15:20:49 INFO | 8 Asesora jurídica Daniela Ospina Giraldo PROVEEDOR 3187742001 Medellín -
15 2026-10-03 15:20:49 INFO | 9 Hermano Santiago Torres Arango FAMILIA 3014456677 La Ceja -
16 2026-10-03 15:20:49 INFO | 10 Línea de emergencias Línea Única de Emergencias EMERGENCIA 123 Nacional -
17 2026-10-03 15:20:49 INFO | Resumen de la agenda del usuario 1 por categoria: PROVEEDOR 3, TRABAJO 3, CLIENTE 1, EMERGENCIA 1, FAMILIA 1, SERVICIO 1
18 2026-10-03 15:20:49 INFO | R - Read: 3 contactos del usuario 1 (categoria PROVEEDOR)
19 2026-10-03 15:20:49 INFO | id apodo nombre completo categoria numero ciudad cuenta Quantum
20 2026-10-03 15:20:49 INFO | 1 Contador Andrés Felipe Restrepo Ospina PROVEEDOR 3104458721 Medellín -
21 2026-10-03 15:20:49 INFO | 7 Soporte de software Nicolás Ramírez Echeverri PROVEEDOR 3126654098 El Retiro -
22 2026-10-03 15:20:49 INFO | 8 Asesora jurídica Daniela Ospina Giraldo PROVEEDOR 3187742001 Medellín -
23 2026-10-03 15:20:49 INFO | U - Update: contacto 1 'Contador' -> 'Contador y revisor fiscal' (1 fila modificada)
24 2026-10-03 15:20:49 INFO | U - Update: contacto 7 actualizado {'numero': '3126654099', 'email': 'mesadeayuda@tecnoretiro.co'}
25 2026-10-03 15:20:49 INFO | U - Update: contacto 4 actualizado {'favorito': 1}
26 2026-10-03 15:23:24 INFO | D - Delete: contacto 2 'Antiguo arrendador' (Jorge Iván Cardona Ruiz, 6045612233) eliminado (1 fila)
27 2026-10-03 15:26:15 INFO | Búsqueda 'Quantum' en la agenda del usuario 1: 3 resultado(s) ['Jefe inmediato', 'Laura - Desarrollo', 'Pedro - Analista']
28 2026-10-03 15:26:15 INFO | Transferencia de 150,000.00 a 'Laura - Desarrollo' registrada en Quantum Core
29 2026-10-03 15:26:15 INFO | Saldo de Ana Torres : 220,000.50 -> 70,000.50
30 2026-10-03 15:26:15 INFO | Saldo de Laura Gomez : 4,349,999.50 -> 4,499,999.50
31 2026-10-03 15:26:15 INFO | Inyeccion SQL en la lectura: leer contactos (conexion, '1 OR 1=1')
32 2026-10-03 15:26:15 INFO | R - Read: 0 contactos del usuario 1 OR 1=1
33 2026-10-03 15:26:15 INFO | -> 0 filas: el texto se trato como un valor, no como codigo SQL
34 2026-10-03 15:26:15 INFO | Búsqueda '1 OR 1=1' en la agenda del usuario 1: 0 resultado(s) []
35 2026-10-03 15:26:15 INFO | -> la busqueda con comillas devolvio 0 filas
36 2026-10-03 15:26:15 WARNING | RECHAZADA Numero con letras -> ValueError: numero no valido '300-ABC': solo digitos, entre 3 y 15
37 2026-10-03 15:26:15 WARNING | RECHAZADA Apodo vacio -> ValueError: el apodo debe tener entre 1 y 40 caracteres
38 2026-10-03 15:26:15 WARNING | RECHAZADA Categoria no permitida -> ValueError: categoria no valida 'AMIGO': use FAMILIA, TRABAJO, PROVEEDOR, CLIENTE, SERVICIO, EMERGENCIA

```

Figura 11. Archivo crud_contactos.log en Visual Studio Code: registro de lo leído (R), lo actualizado (U), lo eliminado (D) y de cada operación rechazada.

Conclusiones y oportunidades de mejora

Las cuatro operaciones del enunciado se cumplieron y quedaron verificadas. Se crearon diez contactos para el usuario 1, se leyeron en consola, el apodo «Contador» cambió a «Contador y revisor fiscal» y el contacto 2, «Antiguo arrendador», fue eliminado. Las capturas del antes y el después y el registro de la consola lo confirman.

Los atributos adicionales convierten la tabla en una agenda útil para el negocio. El nombre completo, la categoría, la empresa y la ciudad permiten identificar a cada contacto sin ambigüedad, filtrar la agenda y resumirla, en lugar de guardar solo un apodo y un número.

La cláusula WHERE es la principal protección de una operación de escritura. Al combinar la llave primaria con el dueño de la agenda, cada actualización o eliminación afecta una sola fila y solo dentro de la agenda que corresponde.

La validación en dos capas hace que la base de datos se proteja a sí misma. Python rechaza los datos inválidos antes de enviarlos, y las restricciones de la tabla rechazan incluso las sentencias que omiten esa validación.

La agenda amplía la funcionalidad de Quantum Core sin duplicar reglas. Las transferencias a un contacto se ejecutan con las mismas funciones del sistema y quedan sujetas a sus controles de saldo, límite y atomicidad.

Oportunidades de mejora

MEJORA	DESCRIPCIÓN
Eliminación lógica	Marcar los contactos como inactivos en lugar de borrarlos, para permitir su recuperación y conservar el historial de transferencias asociadas.
Importación de contactos	Reutilizar el proceso de carga masiva desde Excel para incorporar agendas completas con la misma validación.
Interfaz de usuario	Exponer las funciones CRUD en un menú de consola o una interfaz web para su uso por parte de los clientes.

REPOSITORIO

El código fuente, la base de datos, el registro de operaciones, el diagrama de integración (fuente `.drawio` y exportaciones PNG y SVG), las capturas, el README y este informe se encuentran en github.com/DavisZulu/Semana_7_Act_2.

Referencias

- Castro, E. y de Pablo, C. (2013). *Desarrollo de bases de datos: casos prácticos*. RA-MA.
- DB Browser for SQLite. (2026). *DB Browser for SQLite* [Software]. <https://sqlitebrowser.org>
- Hueso, L. (2012). *Gestión de bases de datos* (2.ª ed.). RA-MA.
- OWASP Foundation. (2026). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Peláez Loaiza, S. (2026). *FUNDAMENTOS-DE-SOFTWARE* [Repositorio de código]. GitHub. <https://github.com/SimonP8/FUNDAMENTOS-DE-SOFTWARE>
- Python Software Foundation. (2026a). *sqlite3: DB-API 2.0 interface for SQLite databases*. The Python Standard Library. <https://docs.python.org/3/library/sqlite3.html>
- Python Software Foundation. (2026b). *logging: Logging facility for Python*. The Python Standard Library. <https://docs.python.org/3/library/logging.html>
- SQLite. (2026a). *CREATE TABLE*. https://www.sqlite.org/lang_createtable.html
- SQLite. (2026b). *SQLite foreign key support*. <https://www.sqlite.org/foreignkeys.html>
- SQLite. (2026c). *SQL language expressions*. https://www.sqlite.org/lang_expr.html
- W3Schools. (2026). *SQL UPDATE statement*. https://www.w3schools.com/sql/sql_update.asp